

TP3 : RNG, 100% des gagnants auront tenté leur chance

Dans ce TP, vous mettrez en place des classes permettant de générer des nombres aléatoires (Random Number Generator), et utiliserez pour cela les surcharges d'opérateurs, les héritages ainsi que les classes abstraites.

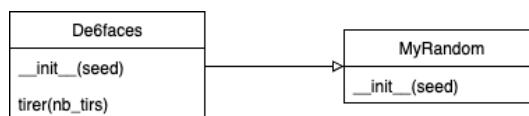
Vous devrez rendre, à la fin du TP, votre travail qui consistera en plusieurs fichiers dont le contenu est expliqué en fin de cet énoncé.

Dans la suite, vous pouvez utiliser votre éditeur de code préféré, mais il est recommandé d'utiliser Pycharm.

1 Random Number Generator

1.1 Première classe : un dé à 6 faces

Nous souhaitons mettre en place une classe permettant de simuler le jet d'un dé à 6 faces. Cette classe héritera d'une classe mère permettant de générer un objet de type **Random**, permettant de générer des nombres aléatoires. A la fin de cette section, votre diagramme de classe devrait ressembler à :



Questions

1. Dans un premier temps, définissez une classe **MyRandom**, possédant une variable de classe de type **Random**. Cette classe **MyRandom** possède un constructeur avec un paramètre **seed** qui vaut, par défaut, **None**. Ce paramètre est utilisé pour initialiser l'objet **Random**.
Pensez à inclure la classe **Random** de la librairie **random** :

Code Python

```
from random import Random
```

2. Créez une classe **De6faces**, qui héritera de **MyRandom** :

Code Python

```
class De6faces(MyRandom):
```

Le constructeur de cette nouvelle classe prend un paramètre **seed** qui vaut, par défaut, **None**. Ce paramètre sera utilisé pour construire l'objet **MyRandom** père à l'aide de la commande :

Code Python

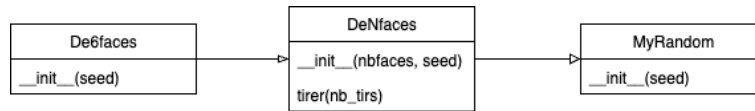
```
super().__init__(seed)
```

3. Dans la classe **De6faces**, nous souhaitons donner à l'utilisateur deux possibilités : tirer un nombre du dé au hasard (entre 1 et 6 donc), et tirer n nombres au hasard, le tout avec une même fonction. Quelle solution adopter pour cela ?

*Indice : regardez la méthode **choices** de la classe **Random**.*

1.2 Un dé à N faces

Nous souhaitons maintenant proposer une seconde classe permettant de simuler un dé à N faces, permettant de tirer aléatoirement un nombre entier entre 1 et N . Si le dé à 6 faces sera souvent utilisé pour émuler des jeux de société assez simples, le dé à N faces sera utile pour faire des tirages aléatoires plus complexes. A la fin de cette section, votre diagramme de classe devrait ressembler à :

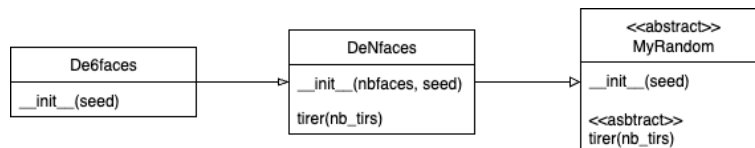


Questions

1. Nous allons commencer par nous interroger sur la façon d'intégrer cette nouvelle classe **DeNfaces** à notre projet, sachant qu'il est tout à fait possible de modifier le code déjà écrit. Commencez pas dessiner le diagramme de classe de votre solution (n'oubliez pas d'adopter une approche "orientée objet"). Dans quelle classe la méthode **tirer** doit-elle être ?
2. Une fois cette solution validée, mettez en place cette solution, en proposant aussi dans votre nouvelle classe **DeNfaces** un moyen de tirer un ou plusieurs nombres aléatoires entre 1 et N . N'oubliez pas non plus de proposer un constructeur dans votre classe prenant en paramètre le nombre de faces du dé.

1.3 Classe abstraite

On souhaite maintenant s'attaquer à un problème de notre conception : la possibilité de générer un **MyRandom**, alors même que cette classe n'est qu'une "coquille presque vide" servant à donner à nos dés un générateur de nombres aléatoires (que l'on améliorera par la suite). A la fin de cette section, votre diagramme de classe devrait ressembler à :



Questions

1. La solution pour empêcher l'instanciation d'un **MyRandom** sera de rendre cette dernière abstraite. Elle ne pourra ainsi pas être directement instanciée, et il sera nécessaire de passer par ses classes filles pour créer un **MyRandom** à travers l'héritage. Pour ce faire, commencez par importer le package Python **abc** (abc signifie ABstract Class) et faites hériter votre classe **MyRandom** de la classe **abc.ABC**.

Tentez maintenant de créer, dans le code principal, un **MyRandom** : est-ce que vous y êtes parvenu ?

2. On peut en effet instancier un **MyRandom** malgré que la classe soit abstraite. Pour empêcher cela, il faut que **MyRandom** possède au moins une méthode abstraite : rendez le constructeur de **MyRandom** abstraite en écrivant au-dessus de sa déclaration :

Code Python

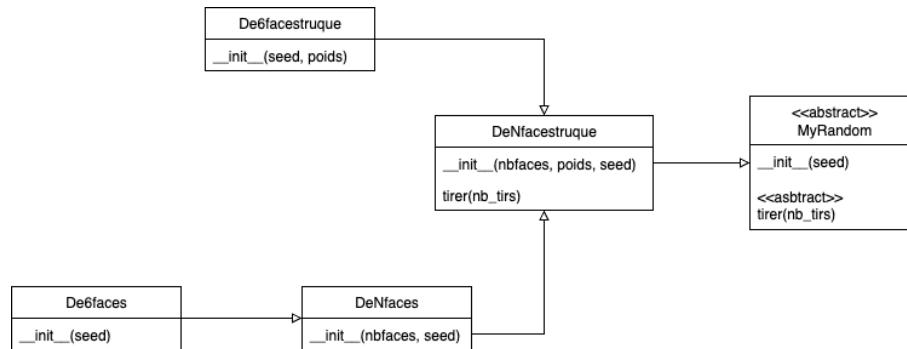
```
@abc.abstractmethod
```

Tentez maintenant de créer, dans le code principal, un **MyRandom** : est-ce que le résultat est différents à précédemment, et conforme à ce à quoi on s'attendait ?

3. Pensez qu'il faudra rendre le travail à la fin de l'heure, et lisez les instructions de rendu à la fin de l'énoncé pour savoir le format de programme qui sera attendu (partie 1).

1.4 Dé truqué

Nous allons maintenant proposer des classes permettant d'instancier des dés truqués, c'est à dire que chaque face n'a pas forcément la même probabilité que les autres d'être sélectionnée. A la fin de cette section, votre diagramme de classe devrait ressembler à :



Questions

1. Commencez par proposer une classe **DeNfacestruque**, héritant de **MyRandom**, qui permettra d'obtenir un dé à n faces truqué. Chaque face aura un poids différent : regardez les détails de la méthode **choices** de **Random** pour comprendre comment mettre en place ces probabilités différentes.

Vous devrez proposer, dans le constructeur, un moyen pour l'utilisateur de spécifier ces probabilités, ou bien de ne pas les spécifier du tout (et dans ce cas, chaque face est équiprobable). Pensez à créer une exception si les poids spécifiés par l'utilisateur ne sont pas en nombre suffisant par rapport au nombre de faces du dé.

N'oubliez pas non plus de coder la méthode abstraite de la classe **MyRandom** dans **DeNfacestruque**.

2. Proposez également une classe **De6facestruque**, de façon "orientée objet".
3. Comment modifier vos classes **De6faces** et **DeNfaces** de façon à ce que votre architecture soit la plus "orientée objet" possible ?
4. Pensez qu'il faudra rendre le travail à la fin de l'heure, et lisez les instructions de rendu à la fin de l'énoncé pour savoir le format de programme qui sera attendu (partie 2).

1.5 Un sac de dés

Dans cette partie, vous mettrez en place une nouvelle classe représentant un paquet de dés (qui n'hérite de personne). Cette classe possèdera une liste de dés, et proposera à l'utilisateur d'ajouter des dés en surchargeant l'opérateur d'addition.

Questions

1. Créez une classe **PaquetDe**, qui contiendra comme variable de classe une liste de dés. Le constructeur de cette classe prendra un nombre variable de paramètres, chacun étant un dé que l'on rangera dans la liste. On souhaite pouvoir appeler le constructeur ainsi :

Code Python

```

de1 = De6faces()
de2 = DeNfacestruque(4, [2, 3, 1, 3])
de3 = DeNfaces(8)

pa = PaquetDe(de1, de2, de3)

```

Pour ce faire, il faut déclarer le constructeur comme prenant une liste d'éléments, avec l'opérateur *splat* devant :

Code Python

```

def __init__(self, *list_de):

```

Ici, **list_de** est un tuple qui contiendra l'ensemble des paramètres transmis au constructeur. Convertissez simplement ce tuple en liste, et rangez cette liste dans une variable de classe de **PaquetDe**.

2. On souhaite maintenant surcharger l'opérateur d'addition, afin de permettre ce type de code :

Code Python

```

de1 = De6faces()
de2 = DeNfacestruque(4, [2, 3, 1, 3])
de3 = DeNfaces(8)
de4 = DeNfaces(12)

pa = PaquetDe(de1, de2, de3)
paquet2 = pa+de4
#On crée un nouveau paquet de dés, qui contiendra de1, de2, de3, et de4

```

Pour ce faire, il faut surcharger, dans notre classe, la fonction `__add__` (attention au double underscore de chaque côté) :

Code Python

```

def __add__(self, other):

```

Le paramètre **self** représente l'objet à gauche du signe d'addition (donc, notre objet en lui-même) tandis que le paramètre **other** représente l'objet à droite de l'addition. Comme nous sommes en train d'écrire le code dans la classe **PaquetDe**, nous avons la garantie que **self** est un objet de type **PaquetDe**.

Voilà les différentes étapes de l'addition :

- (a) Vérifier d'abord que **other** possède le type que l'on attend. Ici, il faut qu'il soit un sous classe de **MyRandom**. Si ce n'est pas le cas, la méthode doit lever une exception de type **TypeError**, avec un message spécifiant le problème.
- (b) Considérez ce code :

Code Python

```

paquet2 = pa+de4

```

On ne s'attend pas, ici, à ce que **pa** soit modifié par l'opération d'addition. Il sera donc nécessaire que l'opérateur "+" ne modifie pas **self**, mais crée un nouvel objet et fasse des opérations dessus. Donc, si **other** possède le bon type, il va falloir créer un nouvel objet de type **PaquetDe** qui contiendra à la fois les dés de **self** et l'objet **other**.

(c) Retourner le nouvel objet ainsi créé.

Une fois votre code écrit, testez-le avec un exemple et vérifiez que l'addition fonctionne comme convenu.

3. Testez maintenant le code suivant :

Code Python

```
de1 = De6faces()
de2 = DeNfacestruque(4, [2, 3, 1, 3])
de3 = DeNfaces(8)
de4 = DeNfaces(12)

pa = PaquetDe(de1, de2, de3)
paquet2 = pa+de4
print(id(de4))
print(id(paquet2.list_de[3]))
```

Les deux entités **de4** et **paquet2.list_de[3]** sont-elles les mêmes ? Si c'est le cas, il s'agit d'un problème car il faut éviter un maximum ce phénomène de dépendances invisibles entre classes. Pour l'éviter, dans votre code de l'addition, il ne faut pas rajouter directement les dés au **PaquetDe** que l'on construit, mais faire une copie de ces dés.

Pour ce faire, regardez la méthode **deepcopy** de la librairie **copy**, et faites en sorte de ne plus avoir de dépendances invisibles entre les éléments des différentes classes.

4. Améliorez votre code pour permettre l'addition de deux **PaquetDe** entre eux : ceci doit avoir pour effet de concaténer les deux listes de dés.
5. Ce code ne doit normalement pas fonctionner :

Code Python

```
paquet2 = de4 + pa
```

Pourquoi ce code ne fonctionne pas ? Que faut-il créer, dans la classe **MyRandom** pour que ce code fonctionne ? Mettez en œuvre votre solution.

6. Proposez une méthode **tirer** dans **PaquetDe** permettant de réaliser un tirage de tous les dés du paquet. Utilisez la compréhension de liste, en une seule ligne de code, pour faire le travail. Votre fonction devrait renvoyer une liste de tirages : le premier élément sera la liste de tous les tirages du premier dé, le second élément sera la liste de tous les tirages du second dé, etc.
7. En réalité, quand on jette un paquet de dé dans un jeu, on s'intéresse à considérer l'ensemble des dés d'un même tirage... Modifiez le plus simplement possible la fonction précédente (pensez à la fonction **zip**) afin qu'elle renvoie comme premier élément le résultat de tous les dés du premier tirage, puis comme second élément le résultat de tous les dés du second tirage, etc.
8. Nous avons fait en sorte que, lors de l'ajout d'un dé à un paquet par la fonction d'addition, nous ne rajoutons que des éléments de héritant de **MyRandom**. Est-on sûr dans ce cas qu'un paquet de dé ne peut contenir que des éléments de type **MyRandom** ? Corrigez votre code, si nécessaire, pour en être sûr.
9. Nous souhaiterions avoir la garantie que tous les éléments que l'on rajoute à un paquet de dé possèdent une méthode de tirage aléatoire. Or, la méthode **tirer** n'est définie que dans la classe des dés truqués... Que modifier dans votre code pour garantir que tout objet héritant de **MyRandom** possèdera une telle fonction ?
10. Voyez-vous un "danger structurel" à faire un héritage entre **PaquetDe** et **MyRandom** ?
11. Pensez qu'il faudra rendre le travail à la fin de l'heure, et lisez les instructions de rendu à la fin de l'énoncé pour savoir le format de programme qui sera attendu (partie 3, la dernière).

1.6 Améliorer nos classes

On souhaite rajouter des fonctionnalités à notre ensemble de classes (sans y passer trop de temps, conformément à l'esprit de Python). A vous de trouver, pour chaque question, la solution qui sera la plus rapide à implémenter.

Questions

1. Proposez une classe **DeNfacesIllustrees** permettant de simuler un dé non truqué à n faces, avec un objet rattaché à chacune des faces. Le constructeur prendra une liste d'éléments en paramètres, et chaque élément sera attribué à une face. Ce dé est différent d'un dé à n faces classique du fait qu'il n'affiche pas une valeur entre 1 et n , mais autre chose (ce peut être un entier, une chaîne de caractères, ...).

Par exemple, nous pourrions créer un dé à 3 faces, chacune étiquetées par une chaîne de caractères représentant l'une des couleurs primaires, en faisant :

Code Python

```
myde = DeNfacesIllustrees("rouge", "bleu", "vert")
```

Lors du lancer d'un tel dé, on n'obtiendra pas une séquence d'entiers, mais une séquences d'objets (dans l'exemple ci-dessus, une séquence de chaînes de caractères).

*Indice : Vous pouvez utiliser un dictionnaire pour mettre en relation des valeurs entières d'un dé classique avec les objets spécifiés par l'utilisateur, et utiliser ainsi la classe **DeNfaces** dans votre code.*

2. Proposez une classe **Piece**, qui vous permettra de simuler le lancer d'une pièce, et affichera, pour chaque lancer, soit pile, soit face.
3. Proposez une classe **SacNBillesSansRemise** représentant un système un peu particulier. Il s'agit d'un sac de billes numérotées de 1 à n . A chaque tirage, on choisit une bille au hasard et on ne la remet pas dans le sac : la valeur correspondant à la bille ne pourra plus être tirée. Une fois que le sac est vidé, on remet toutes les billes dans le sac.

*Indice : pour implémenter ce type de classe, vous pouvez imaginer générer dès début les valeurs qui seront tirées à l'aide d'une permutation aléatoire des entiers de 1 à n . Regardez la fonction **sample** de la classe **Random**.*

2 Facultatif : améliorer le système de génération de nombres aléatoires

Le but de toutes les classes de l'exercice précédent est de générer des nombres aléatoires, il est donc important que notre système de génération de nombres aléatoires soit solide. Dans le cas où la graine n'est pas spécifié pour un **De**, le système prend en compte l'heure actuelle pour sa génération de nombre aléatoire. Nous allons améliorer ce principe en utilisant un système externe pour générer des nombres aléatoires.

Questions

1. Regardez le site web <https://quantumnumbers.anu.edu.au/>, et lisez l'explication sur le système utilisé pour générer des nombres aléatoires. Lisez ensuite la documentation de l'API (<https://quantumnumbers.anu.edu.au/documentation>) pour comprendre comment accéder aux nombres aléatoires.

Nous tenterons, dans la suite, d'utiliser ce site pour récupérer des nombres aléatoires qui nous serviront de graine.

2. Pour effectuer cette tâche, commencez par regarder les fichiers de ce projet : <https://github.com/lmacken/quantumrandom> (les fichiers ont été dupliqués et sont disponibles sur Moodle au cas où). Dans le dossier **quantumrandom**, regardez le fichier `__init.py__`, et cherchez dedans les lignes de code où l'accès au site est réalisé. Vous devrez certainement utiliser une clef d'API que vous aurez obtenue du site pour réaliser cette opération.

3. A partir de là, développez votre propre moyen d'accéder au site qrng et d'en récupérer un nombre aléatoires. Proposez, dans le constructeur de la classe **De**, d'initialiser le **seed**, dans le cas où il n'a pas eu de valeurs spécifiques, avec la valeur récupérée du site. Dans le cas où le site ne répondrait pas (regardez du côté des exceptions comment faire), faites comme avant et construisez un objet **Random** sans paramètre.
4. Testez votre classe pour vérifier que votre programme arrive à se connecter au site et arrive à récupérer des nombres.

3 Rendu du programme

Vous devrez ensuite rendre tous vos fichiers python, **sauf le dossier venv**, dans un fichier zip dont le nom aura comme format

Code Python

```
VotreNom_VotrePrenom_Votre NumeroEtudiant.zip
```

sans espace ni accent.

Vous devrez rendre plusieurs programmes :

1. Un programme permettant de générer un ou plusieurs nombres aléatoires, à l'aide d'une dé. Votre programme devra s'exécuter avec la commande :

Code Python

```
python3 denface.py nb_face nb_tirages
```

Le programme devra construire un dé comme demandé par l'utilisateur (De6faces ou DeNfaces), et générer **nb_tirages** tirages aléatoires et afficher le résultat de chacun d'eux. Les résultats devront s'afficher à l'écran, un par ligne, et aucune autre information ne devra s'afficher. A la fin, le programme devra se terminer avec le code de retour 0 (**exit(0)**).

Par exemple, ce code

Code Python

```
python3 denface.py 9 3
```

pourrait générer ce résultat :

Code Python

```
5
9
2
```

2. Un programme permettant de générer un ou plusieurs nombres aléatoires, à l'aide d'une dé dont on spécifiera les poids de chaque face. Votre programme devra s'exécuter avec la commande :

Code Python

```
python3 denfacetruque.py nb_face nb_tirages poids_face_1 poids_face_2 ...
                                poids_derniere_face
```

Le programme devra construire un dé truqué comme demandé par l'utilisateur (De6facesTruque ou DeNfacesTruqué), avec les poids spécifiés et générer **nb_tirages** tirages aléatoires et afficher le résultat de chacun d'eux. Les résultats devront s'afficher à l'écran, un par ligne, et aucune autre information ne devra s'afficher. A la fin, le programme devra se terminer avec le code de retour 0 (**exit(0)**).

Par exemple, pour construire un dé à 6 faces truqué où les faces paires auront deux fois plus de chances d'être tirées que les faces impaires, et effectuer 8 tirages, on fera

Code Python

```
python3 denfacetruque.py 6 8 1 2 1 2 1 2
```

pourrait générer ce résultat :

Code Python

```
2
4
1
6
3
1
6
2
```

3. Un programme permettant de générer des ensembles de nombres aléatoires, à l'aide d'un paquet de dés non truqués dont on spécifiera la composition. Votre programme devra s'exécuter avec la commande :

Code Python

```
python3 paquetde.py nb_tirages nb_face_de1 nb_face_de2 ... nb_face_dernier_de
```

Le programme devra construire un paquet de dés composé des dés demandés, et effectuer **nb_tirages** aléatoires et afficher le résultat de chacun d'eux. Les résultats devront s'afficher à l'écran, un par ligne, et aucune autre information ne devra s'afficher. A la fin, le programme devra se terminer avec le code de retour 0 (**exit(0)**).

Par exemple, pour construire un paquet composé de deux dés à 6 faces et un dé à 9 face, et effectuer 5 tirages aléatoires, on fera

Code Python

```
python3 paquetde.py 5 6 6 9
```

pourrait générer ce résultat :

Code Python

```
4 1 7
2 2 4
5 6 5
6 4 2
3 2 8
```

3.1 JARVIS

Pour vous aider dans votre travail, vous pouvez utiliser JARVIS. Pour tester immédiatement votre programme, préparez votre fichier comme expliqué ci-dessus. Puis, envoyez le à l'adresse :

```
chaussard.laga+pytp3_nface@gmail.com
```


pour le travail sur les dés n faces ou bien

```
chaussard.laga+pytp3_truque@gmail.com
```

pour le travail sur les dés truqués ou bien

```
chaussard.laga+pytp3_paquet@gmail.com
```

pour le travail sur les paquets de dés.

Deux minutes plus tard, vous devriez recevoir un mail de JARVIS qui aura testé votre programme et vous transmettra les résultats de ses tests. Votre message n'a pas besoin d'avoir de sujet particulier ou bien de message particulier, simplement votre programme en fichier joint. N'incluez qu'un seul fichier avec votre mail (un fichier compressé selon les instructions précédentes).